Tutorial: GBM Options Pricing & RL Trading Agent

Nov 7, 2026 · @D

Overview & roadmap

`gbm_options_rl_agent.ipynb` prices 1-month SPY options with geometric Brownian motion (GBM), forecasts volatility, and trains a deep Q-network (DQN) agent to buy and sell calls and puts. It then tests that agent out-of-sample against simple benchmarks. This tutorial walks through it end to end: the mathematics first, then why the code is shaped as it is, then every code cell in order.

The notebook is one pipeline. Each stage feeds the next:

1. **Data**: SPY prices, VIX (the implied-vol proxy) and the 13-week T-bill rate.
2. **GBM model**: fit μ and σ, and check where GBM's assumptions break.
3. **Volatility forecasts**: five models, scored out-of-sample on the next 21 days.
4. **Pricing engine**: Black-Scholes, Greeks, and Monte Carlo checked against each other.
5. **GBM forecasts**: a price fan chart and a calibration test.
6. **GBM edge**: is each option cheap or expensive relative to what GBM expects it to pay?
7. **RL agent**: a DQN that runs an options book with spreads, margin and a crash filter.
8. **Backtest & tracking**: test-period results, benchmarks, and a log of every trade.

Cell map. The notebook has 21 code cells. Each part of this tutorial covers its own cells.

Code cell	Notebook section	What it does	Tutorial part
1–2	Setup	Imports, func_lib, seeds, configuration	Part 1
3–4	1. Data	Download SPY, VIX, rates; price and IV chart	Part 1
5	2. GBM fit	MLE of μ, σ; histogram and Q-Q plot	Part 1
6–7	3. Vol forecasting	Five forecasts, accuracy table, VRP	Part 2
8–9	4. Pricing engine	MC convergence; Greeks vs spot	Part 3
10–12	5. GBM forecasts	Drift, fan chart, today's view, calibration	Part 4
13	6. Option features	Market price vs GBM value: the edge	Part 5

Code cell	Notebook section	What it does	Tutorial part
14–15	7. RL agent	States, agent, training loop	Parts 6–7
16–17	8. Test	Greedy test, benchmarks, equity and stats	Part 9
18–20	9. Tracking	Trade statistics, trade log, dashboards	Part 9
21	10. Today's signal	Q-values for today, invalid actions masked	Part 9

How to read it. Each part opens with the idea and its formulas, then shows the code cells that implement it, with a worked example. Part 8 explains how the code is organized across the notebook and `func_lib.py`. Part 10 sets out next steps for the agent and for the pricing and forecasting models. Part 11 collects exercises, common pitfalls and references.

What you need: basic probability (normal distribution, expectation), comfort reading pandas and NumPy, and the notebook open beside this doc. No prior options or reinforcement-learning knowledge is assumed.

Part 1 — Setup, data and the GBM model (cells 1–5)

GBM says a stock's log returns are normal and independent, with constant drift μ and volatility σ. Everything downstream (pricing, forecasts, the edge signal) rests on this one assumption. Part 1 also shows where it fails.

The model

The price follows a stochastic differential equation. Each instant, the price changes by a deterministic drift plus a random shock that scales with the price:

```
dS_t = \mu S_t\,dt + \sigma S_t\,dW_t
```

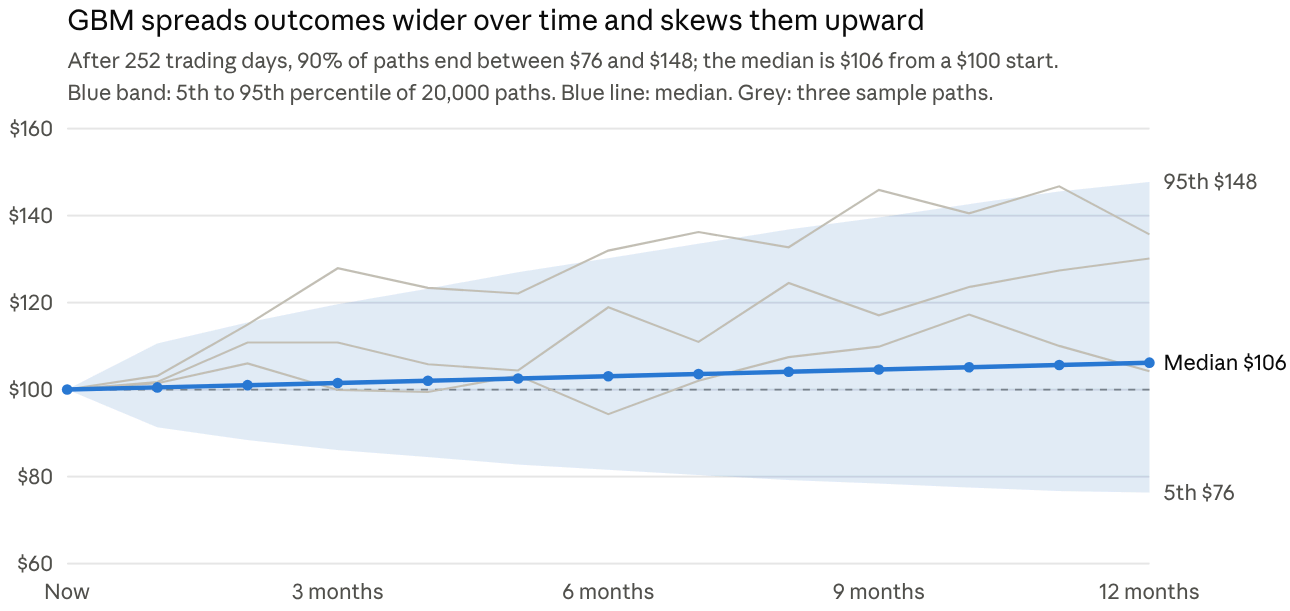
W is a Brownian motion: its increments over a step dt are normal with mean 0 and variance dt. Applying Itô's lemma to log S gives a solution in closed form:

```
S_T = S_0 \exp\!\Big(\big(\mu - \tfrac{1}{2}\sigma^2\big)T +
\sigma\sqrt{T}\,Z\Big), \qquad Z \sim N(0,1)
```

So the log return over T is normal, and the price itself is **lognormal**: it can never go negative, and its distribution is skewed right. Two facts follow that the code relies on:

$$\ln\frac{S_T}{S_0} \sim N\Big(\big(\mu-\tfrac{1}{2}\sigma^2\big)T,\; \sigma^2 T\Big), \qquad \mathbb{E}[S_T] = S_0\,e^{\mu T}$$

The $-\frac{1}{2}\sigma^2$ term is the **volatility drag**. The *median* path grows at $\mu - \frac{1}{2}\sigma^2$, while the *mean* grows at μ. With $\mu = 8\%$ and $\sigma = 20\%$, the median grows at 6% a year. Volatility spreads the outcomes, and the bad ones compound.



Simulated with func_lib.gbm_paths · 20,000 paths, S0 = 100, μ = 8%, σ = 20%, 252 trading days, seed 7

The band widens like $\sqrt{t}$, and it is lopsided: the 95th percentile sits $42 above the median, the 5th only $30 below. That asymmetry is the lognormal shape.

Estimating μ and σ (maximum likelihood)

Because daily log returns r_t are i.i.d. normal under GBM, the maximum-likelihood estimates are simply their sample mean and standard deviation, annualized with 252 trading days:

$$\hat\sigma = \mathrm{std}(r_t)\,\sqrt{252}, \qquad \hat\mu = 252\,\overline{r_t} + \tfrac{1}{2}\hat\sigma^2$$

The $+\frac{1}{2}\sigma^2$ converts the mean *log* return back into the drift μ of the SDE.

Where GBM breaks

Real equity returns have **fat tails** (excess kurtosis well above the normal's 0) and **negative skew**: crashes are larger and more frequent than a normal allows. Volatility is also not constant: it clusters, and it is "rough" (Gatheral et al., 2018). The notebook keeps GBM because it is tractable and gives Black-Scholes. It then measures the cost of the assumption: Part 2 forecasts σ day by day instead of holding it fixed, and Part 4 tests how often GBM's forecast ranges are breached.

The cells

Cell 1: imports and func_lib. Standard imports, then a loop that adds the notebook's folder or its parent to `sys.path` if `func_lib.py` is there. Importing `func_lib` also registers the dark Plotly theme. The cell pulls in the functions it uses by name, sets `SEED = 42` through `keras.utils.set_random_seed` (which seeds Python, NumPy and the backend together), and turns off Keras's per-call progress bars.

Cell 2: configuration. Every tunable number lives here, so an experiment is a one-line change:

- data: `TICKER='SPY'`, `IV_TICKER='^VIX'`, `RF_TICKER='^IRX'`, dates
- forecasting: `FIRST_FORECAST_DATE='2010-01-01'`, `HORIZON=21` (forecast horizon = option tenor), `EWMA_LAMBDA=0.94`, `MU_LOOKBACK=252`, `MU_SHRINK=0.5`
- the RL book: `OPTION_CFG` (tenor, moneyness, `cost_vol=0.01`, capital, position caps, margin rates) and `AGENT_PARAMS`
- benchmarks and risk: `EDGE_THRESHOLD`, `SELL_EDGE_THRESHOLD`, `PANIC_FILTER`

Cell 3: data. `func_lib.download_ohlcv` fetches SPY's adjusted OHLCV with flat columns. `func_lib.download_close` fetches VIX and the T-bill yield, which are left-joined onto SPY's dates and forward-filled across holiday mismatches. Two derived columns follow: `r` = the T-bill yield as a decimal, and `log_ret = log(Close).diff()`. Rows missing any input are dropped.

Cell 4: chart. Two stacked panels: SPY's price, and VIX shaded underneath. VIX spikes (2008, 2020, 2022) are the regimes where GBM's constant-volatility assumption is most wrong. And also note those spikes are consistent with market crashes.

Cell 5: GBM fit and diagnostics. It computes $\hat{\sigma}$ and $\hat{\mu}$ with the formulas above, and prints skewness and excess kurtosis. Then it draws two panels:

- a histogram of daily log returns with the fitted normal density on top: the empirical peak is taller and the tails fatter
- a normal Q-Q plot: sorted standardized returns against normal quantiles `norm.ppf((i - 0.5)/n)`. Points that bend away from the 45° line at both ends are the fat tails.

```
sigma_hat = log_ret.std() * np.sqrt(TRADING_DAYS)
mu_hat = log_ret.mean() * TRADING_DAYS + 0.5 * sigma_hat ** 2
z = np.sort(((log_ret - log_ret.mean()) / log_ret.std()).to_numpy())
theoretical = norm.ppf((np.arange(1, len(z) + 1) - 0.5) / len(z))
```

Part 2 — Volatility forecasting (cells 6–7)

GBM's σ is the one input that moves an option price most, so the notebook forecasts it fresh every day instead of using one historical number. Five forecasts compete on the same target: the average daily variance over the **next 21 trading days**, the life of the options traded.

The target

```
y_t = \frac{1}{21}\sum_{k=1}^{21} r_{t+k}^2
```

In code it is `forward_realized_variance(r, 21)`, which is `(r**2).rolling(21).mean().shift(-21)`: the rolling mean ending at t+21, moved back to row t. Every forecast at t may use data up to and including day t, and never y_t itself.

A better daily variance input: range-based realized variance

A squared daily return is a very noisy estimate of that day's variance. The day's high, low, open and close carry more information. The notebook adds the squared overnight gap to the Garman-Klass intraday estimator:

```
RV_t = \ln^2\!\frac{O_t}{C_{t-1}} + \tfrac12\ln^2\!\frac{H_t}{L_t} - (2\ln 2
- 1)\ln^2\!\frac{C_t}{O_t}
```

This is `daily_variance_proxy(ohlc)`. HAR uses it as input. The target stays close-to-close, so all models are scored on the same thing.

The five forecasters

Model	Forecast of daily variance at t	Code
Historical 21d	mean of r^2 over the last 21 days	`(r**2).rolling(21).mean()`
EWMA (RiskMetrics)	$\sigma^2_t = \lambda\sigma^2_{t-1} + (1-\lambda)r^2_t$, with $\lambda = 0.94$	`(r**2).ewm(alpha=0.06, adjust=False).mean()`
GARCH(1,1)	MLE fit; 21-day average of the multi-step variance path	`walk_forward_garch`
HAR-RV	OLS on daily, weekly and monthly averages of RV	`walk_forward_har`
Implied (VIX)	$(\text{VIX}/100)^2/252$	the market's own forecast

GARCH(1,1) makes tomorrow's variance a weighted blend of a long-run level, today's squared return and today's variance:

```
h_{t+1} = \omega + \alpha\,r_t^2 + \beta\,h_t, \qquad V_L = \frac{\omega}{1-\alpha-\beta}
```

The parameters come from maximizing the Gaussian log-likelihood, written $-\tfrac{1}{2}\Sigma(\log h_t + r^2_t/h_t)$, under $\alpha + \beta < 1$. Forecasts revert to V_L geometrically at rate $\varphi = \alpha + \beta$, so the average over the next H days has a closed form:

```
\bar h_{t,H} = V_L + (h_{t+1} - V_L)\,\frac{1-\phi^H}{(1-\phi)\,H}
```

The **Heterogeneous Autoregressive model of Realized Volatility (HAR-RV)** captures volatility's long memory with three averages that stand for daily, weekly and monthly traders: (Corsi, 2009)

```
y_t = \beta_0 + \beta_d RV_t + \beta_w \overline{RV}_{t-4:t} + \beta_m \overline{RV}_{t-21:t} + \varepsilon_t
```

HAR is the standard strong baseline, and that "rough" volatility models can beat it.

Walk-forward fitting: why and how

A model fitted on the whole sample and scored on the same sample knows the future. Both GARCH and HAR are therefore **refitted every January**, using only data known by then:

- GARCH: fit on all returns before the refit date. Then forecast for that year with those fixed parameters (filtering at t uses only returns up to t).
- HAR: a training row t is used only if its target window t+1…t+21 ended before the refit date: `train = slice(0, start - horizon)`. Without the `- horizon`, the last 21 training targets would contain returns from the test year.

Design note. The GARCH variance recursion is a first-order linear filter, so it runs vectorized through `scipy.signal.lfilter([1], [1, -β], ω + αr²)` with an initial condition, rather than a Python loop. The likelihood is evaluated hundreds of times per fit, over 15+ annual refits, so this matters.

Scoring: QLIKE and friends

`forecast_metrics` reports RMSE, MAE and bias in annualized vol points; **QLIKE**; and the Mincer-Zarnowitz R^2 (the squared correlation of forecast with outcome).

```
\mathrm{QLIKE} = \frac{1}{n}\sum_t \left(\frac{y_t}{f_t} - \ln\frac{y_t}{f_t}
- 1\right)
```

QLIKE is 0 for a perfect forecast and penalizes *under*-prediction of variance more than over-prediction. That is the right asymmetry for an options seller, and it is robust to noise in the proxy (Patton, 2011). The table is sorted by it.

The cells

Cell 6 builds the RV proxy and the target, runs the five forecasters into a dict `var_forecasts`, and keeps only out-of-sample dates (from `FIRST_FORECAST_DATE`). It then draws the metrics table with `stats_table_figure`.

Cell 7 converts each variance to annualized vol, $\sqrt{f \times 252}$, and plots HAR, GARCH and VIX against the realized forward vol. It then measures the **variance risk premium** (VRP): implied minus subsequently realized vol. It prints the share of days that VRP was positive, and plots its distribution.

What to expect: HAR and GARCH beat historical vol on QLIKE. VIX tracks realized vol closely (high R^2) but sits above it on most days (a positive bias). That bias is the premium option buyers pay for insurance, and it is the economic fact the trading agent is up against: buying options at implied vol usually overpays.

Part 3 — Pricing: Black-Scholes, Greeks and Monte Carlo (cells 8–9)

An option's fair price is its expected payoff under the **risk-neutral** measure, discounted at the risk-free rate. Under GBM that expectation has a closed form, Black-Scholes. Monte Carlo reaches the same number by simulation, and the notebook checks one against the other.

Risk-neutral pricing

A hedged option position earns the risk-free rate, so for pricing the stock's real drift μ is replaced by r (less any dividend yield q):

```
V_0 = e^{-rT}\,\mathbb{E}^{\mathbb{Q}}\big[\text{payoff}(S_T)\big], \qquad
dS_t = (r - q)S_t\,dt + \sigma S_t\,dW_t^{\mathbb{Q}}
```

This is why μ never appears in a price. It reappears only in Part 5, where we ask what an option is *expected* to pay in the real world.

Black-Scholes

```
C = S e^{-qT} N(d_1) - K e^{-rT} N(d_2), \qquad P = K e^{-rT} N(-d_2) - S
e^{-qT} N(-d_1)
```

```
d_1 = \frac{\ln(S/K) + (r - q + \tfrac12\sigma^2)T}{\sigma\sqrt T}, \qquad
d_2 = d_1 - \sigma\sqrt T
```

$N(d_2)$ is the risk-neutral probability that the call ends in the money. **Put-call parity**, $C - P = Se^{-gT} - Ke^{-rT}$, holds exactly in `bs_price`, and was verified to machine precision.

Worked example, used throughout: $S = K = 100$ (at the money), $T = 21/252$ (one month), $r = 4\%$, $\sigma = 20\%$, $q = 0$.

Quantity	Call	Put
Price	$2.4694	$2.1366
Delta (per $1 of spot)	0.5345	−0.4655
Gamma (per $1)	0.0688	0.0688
Vega (per 1 vol point)	$0.1147	$0.1147
Theta (per calendar day)	−$0.0433	−$0.0324
Rho (per 1% rate)	$0.0425	−$0.0406

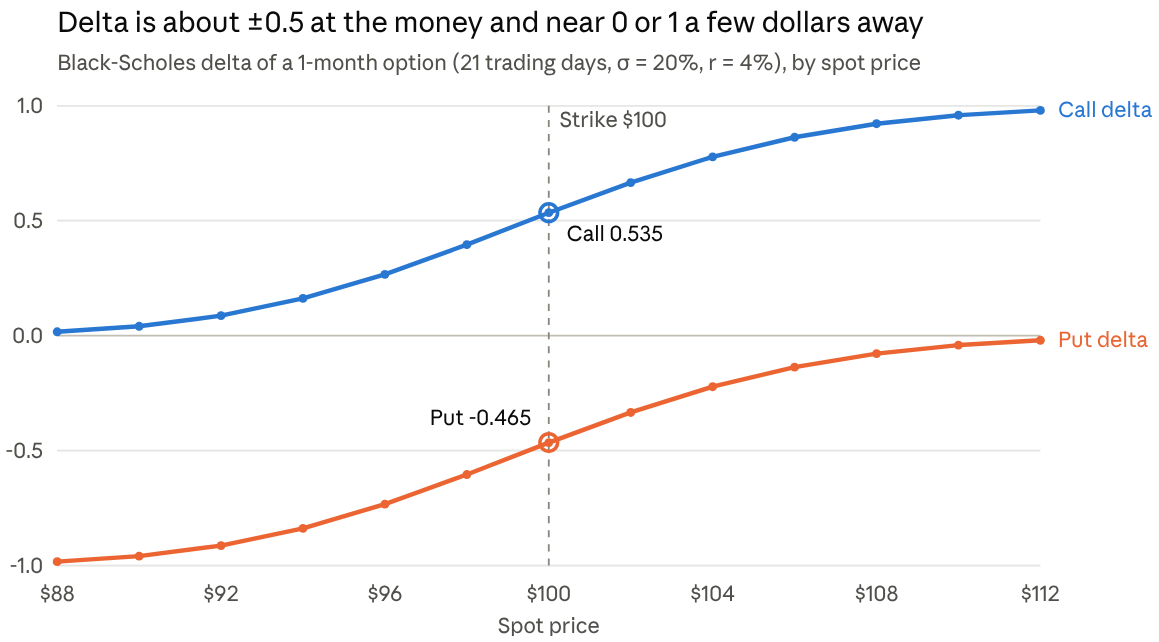
The spread the agent pays shows up in these numbers. Buying at IV + 1 vol point costs $2.5841. Selling at IV − 1 point fetches $2.3546. The $0.23 gap is two vegas, about 9% of the option's value per round trip.

The Greeks

Greeks are the price's sensitivities, and `bs_greeks` returns all five, vectorized:

```
\Delta_C = e^{-qT}N(d_1), \quad \Gamma = \frac{e^{-qT}\varphi(d_1)}
{S\sigma\sqrt T}, \quad \mathcal{V} = S e^{-qT}\varphi(d_1)\sqrt T
```

φ is the standard normal density. The code reports vega per 1 vol point (÷ 100), theta per calendar day (÷ 365) and rho per 1% rate move (÷ 100): the units a trader reads. Every Greek was checked against a finite-difference bump of `bs_price` to six decimals. For short positions, the trade log stores **position** Greeks: the option's Greek times −1, so a short call shows negative delta and positive theta.



Computed with func_lib.bs_greeks · K = 100, T = 21/252, r = 4%, σ = 20%, q = 0

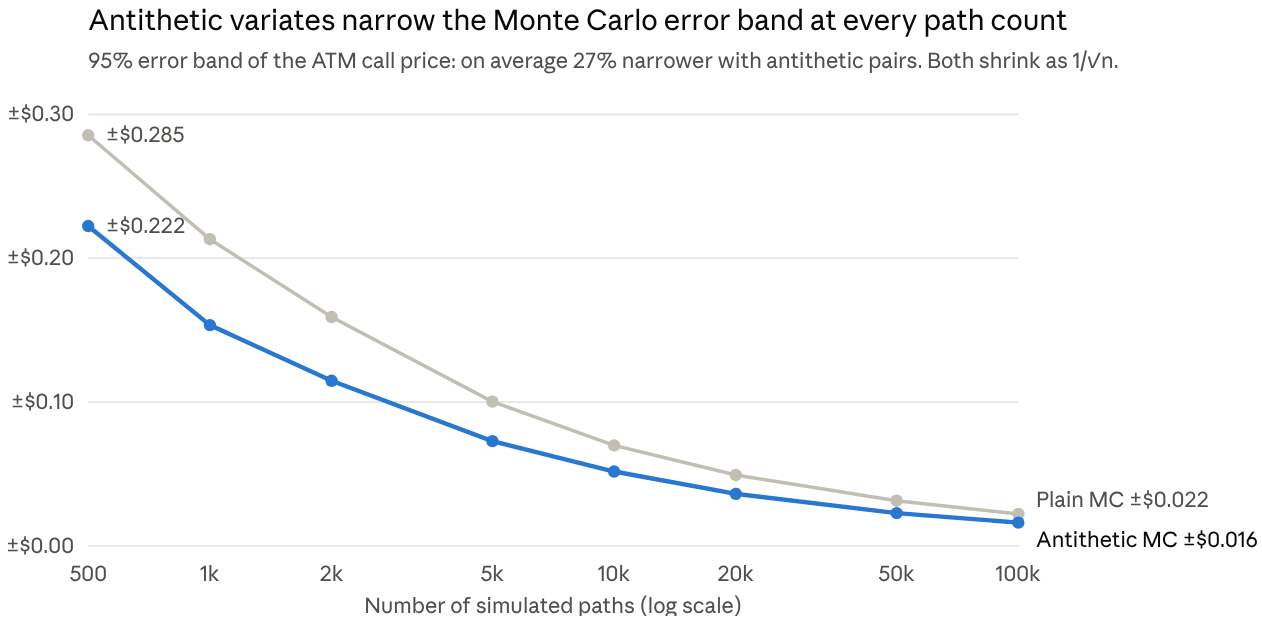
Call and put deltas always differ by exactly 1 (put-call parity). The curve is steepest at the strike, and its slope there is gamma.

Monte Carlo under GBM

Simulate terminal prices exactly in log space (no time-stepping error), average the payoffs, and discount:

```
\hat V = e^{-rT}\frac{1}{n}\sum_{i=1}^{n}\text{payoff}\Big(S_0 e^{(r-q-
\frac12\sigma^2)T + \sigma\sqrt T Z_i}\Big), \qquad \mathrm{SE} = \frac{s}
{\sqrt n}
```

The error falls only as 1/√n: four times the paths halve it. **Antithetic variates** reuse each draw Z as −Z too. The pair's payoffs are negatively correlated, so their average varies less. The standard error must be computed on the pair averages, not on 2n separate draws, or it is understated. `mc_option_price` does this. In the example the 95% error band is about 27% narrower at every path count, which is worth roughly 1.9× as many plain paths.



Computed with func_lib.mc_option_price · ATM call, S = K = 100, T = 21/252, r = 4%, σ = 20%, seed 1; Black-Scholes price $2.4694

`mc_option_price(..., drift=μ)` swaps the risk-neutral drift for a real-world one. It then returns the expected payoff under that GBM, discounted at r. Part 5 uses the closed-form version of this.

The cells

Cell 8 takes today's inputs: spot S0 = last close, r0, and σ = the latest HAR forecast, with K0 = S0 and T0 = 21/252. It prices the call with Black-Scholes. It then runs Monte Carlo at nine path counts from 500 to 200,000, plain and antithetic, building one tidy DataFrame `conv`. The chart shows each estimate with its 95% band (two invisible traces, the second filled `tonexty` to the first) and the Black-Scholes price as a dashed line. The bands should close in on that line.

Cell 9 evaluates `bs_greeks` on 200 spot prices from 85% to 115% of S0, for a call and a put. It draws delta, gamma, vega and theta in a 2×2 grid with the strike marked. Things to notice: delta's S-curve; gamma and vega peaking at the money and identical for calls and puts; theta most negative at the money, where time value is largest.

Part 4 — Forecasting with GBM: drift, fan chart, calibration (cells 10–12)

To *forecast* rather than price, GBM needs the real-world drift μ. The notebook builds it from momentum, simulates paths, and then checks honestly how often reality lands inside GBM's forecast ranges.

A drift you can defend

Mean returns are notoriously hard to estimate: a 12-month average of SPY swings by tens of percent. The notebook shrinks it halfway toward the risk-free rate:

```
\mu_t = s\big(252\,\overline{r}_{t-251:t} + \tfrac12\sigma_{f,t}^2\big) + (1-s)\,r_{f,t}, \qquad s = \texttt{MU\_SHRINK} = 0.5
```

The trailing term is time-series momentum (Moskowitz, Ooi & Pedersen): trends persist over 1 to 12 months. σ_f is the HAR forecast, and +½σ² converts the mean log return to a drift. Setting s = 0 gives a risk-neutral, no-view forecast. Setting s = 1 trusts momentum fully.

Calibration: does reality land in GBM's ranges?

For each non-overlapping 21-day window starting at t, standardize the realized log return with that day's forecast:

```
z_t = \frac{\ln(S_{t+21}/S_t) - (\mu_t - \tfrac12\sigma_t^2)\tau}{\sigma_t\sqrt{\tau}}, \qquad \tau = 21/252
```

If GBM with that σ is right, z is standard normal. Then 50% of windows fall inside |z| < 0.674, 90% inside |z| < 1.645, and 5% in each tail. The **probability integral transform** u = N(z) should be uniform: a flat histogram at 10% per decile. Typical failures:

- too few windows in the 90% range → the σ input is too low
- too many in the lower 5% tail → crash risk GBM does not model
- a U-shaped PIT histogram → ranges too narrow; a hump → too wide (VIX tends to produce this, since it overstates σ)

Non-overlapping windows (`df.index[oos][::HORIZON]`) matter. Overlapping 21-day returns share 20 days each, which would make the counts look far more precise than they are.

The cells

Cell 10: fan chart. It builds `mu_t` with the formula above, then calls `gbm_paths(S0, μ0, σ0, 63/252, 63, 20000)` for a 63-day forecast. That function simulates the log increments for all paths in one array, uses antithetic pairs, and cumulative-sums down the time axis. The chart shows the last six months of history, the 5–95% and 25–75% bands, 30 faint sample paths and the median. Part 1's chart shows the same construction over a year.

Cell 11: today's view. For today's ATM 1-month call and put, it compares:

- the market price: Black-Scholes at implied vol (VIX)
- the GBM forecast value: `gbm_expected_value` with μ_t and HAR vol (Part 5)
- the edge, P(ITM) from 20,000 simulated terminal prices, and delta, vega and theta at implied vol

The result is drawn as a dark table with `stats_table_figure`.

Cell 12: calibration. It computes z for every vol input and window, then tabulates coverage of the 50% and 90% ranges and each 5% tail. It also overlays the PIT histograms for HAR and VIX against the 10% line. This is the notebook's honesty check. Read it before trusting any GBM-based edge.

Part 5 — The GBM edge signal (cell 13)

The edge asks one question of each option: does the market charge more or less than GBM expects it to pay? It uses the real-world drift μ and our own volatility forecast, against the market's implied vol. A positive edge means the option looks cheap.

Real-world expected value, in closed form

We want the expected payoff under the real-world GBM (drift μ, vol σ_f), discounted at the risk-free rate r:

```
G = e^{-rT}\,\mathbb{E}^{\mu}\big[(S_T - K)^+\big]
```

No new formula is needed. Black-Scholes with the rate set to μ computes exactly e^{−μT} times this expectation under drift μ. So the expectation is e^{μT} times that Black-Scholes value, and discounting at r gives:

```
G = e^{(\mu - r)T}\;\mathrm{BS}(S, K, T,\ \text{rate} = \mu,\ \sigma_f),
\qquad \text{edge} = \frac{G}{\mathrm{BS}(S,K,T,r,\sigma_{IV})} - 1
```

That is `gbm_expected_value`. It was checked against Monte Carlo with `drift=μ`, and the two agree within the simulation error.

Worked example: where the edge comes from

An ATM 1-month option with S = K = 100, r = 4%, implied vol 18%, HAR forecast 15%, drift μ = 10%:

	Market price (IV 18%)	GBM value	Edge	Edge with μ = r (vol effect only)
Call	$2.2399	$2.1777	−2.8%	−15.4%
Put	$1.9071	$1.3437	−29.5%	−18.0%

Two forces add up:

1. **Volatility (the variance risk premium).** Implied 18% vs forecast 15% makes *both* options look about 15–18% overpriced. This is the usual state of the market.
2. **Drift.** A positive μ tilts the expected terminal price upward, worth about +12.6 points to the call and −11.5 to the put.

So with a strong trend the call can look roughly fairly priced while the put looks very expensive. **That makes the edge mostly a view on μ and on the VRP**, which is why `MU_SHRINK` is the single most important setting in the notebook.

Two cautions:

- GBM understates crash risk (Part 4). A put that "looks 30% expensive" is partly insurance against a tail GBM does not model. Selling it harvests the premium, but at the cost of rare large losses.
- An expectation is not a forecast of any single trade. Most ATM options expire worth little; their value comes from the few that pay off.

The cell

Cell 13 assembles one feature frame `opt` with:

- spot and rate
- HAR vol `sigma_f` and the drift `mu`

- SMA_20, SMA_50, RSI and Bollinger Bands from `func_lib.build_ts_features`
- `iv` = VIX/100, or HAR × 1.10 when `IV_TICKER = None`
- `vrp = iv − sigma_f`

The two edges are computed vectorized: `bs_price` and `gbm_expected_value` accept whole pandas Series, so a single call prices every day. Rows before `FIRST_FORECAST_DATE` or with any missing value are dropped. The chart plots both edges over time with the rule threshold (+5%), and the cell prints how often each edge clears it.

```
for kind in ('call', 'put'):
    market = bs_price(opt['Close'], opt['Close'], tau, opt['r'], opt['iv'],
kind=kind)
    model = gbm_expected_value(opt['Close'], opt['Close'], tau, opt['r'],
opt['mu'], opt['sigma_f'], kind)
    opt[f'{kind}_edge'] = model / market - 1
```

Part 6 — Reinforcement learning: from Q-learning to DQN

The agent learns a **Q-function**: for each market-and-book state, the expected discounted future reward of each action. It then acts greedily on it. A neural network approximates Q, trained on replayed past experience. This part is the theory behind `func_lib.Agent`.

The decision problem (an MDP)

Element	In this notebook
State s_t	market features (changes and levels) + book features (positions held, time left, margin use)
Action a_t	hold, buy call, buy put, sell call, sell put, close oldest
Reward R_{t+1}	change in account equity from t to t+1, as % of starting capital
Transition	the market moves one day; options are re-marked, expire, or are margin-called
Discount γ	0.95

The agent maximizes the discounted return $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$ With $\gamma = 0.95$ the effective horizon is about $1/(1-\gamma) = 20$ days, close to the 21-day option life. The agent therefore weighs a trade's whole lifetime, not just the next day.

Bellman optimality and Q-learning

The optimal Q-function satisfies a self-consistency equation:

```
Q^*(s,a) = \mathbb{E}\Big[R_{t+1} + \gamma \max_{a'} Q^*(s_{t+1}, a')
\,\Big|\, s_t = s,\ a_t = a\Big]
```

Q-learning turns it into an update: move Q(s, a) toward the **target** $y = r + \gamma \max_{a'} Q(s', a')$. At the end of an episode (`done`) there is no future, so $y = r$.

Deep Q-networks (DQN)

With continuous states a table is impossible, so a network Q_θ maps a state to one Q-value per action. In `build_dqn` that network is Dense(32, ReLU) → Dense(8, ReLU) → linear output, trained by minimizing squared Bellman error with Adam:

```
L(\theta) = \frac{1}{B}\sum_{i=1}^{B}\Big(r_i + \gamma\,(1-
d_i)\max_{a'}Q_{\bar\theta}(s_i', a') - Q_\theta(s_i, a_i)\Big)^2
```

Three ingredients make this stable enough to work:

1. **Experience replay.** Transitions (s, a, r, s', done) go into a buffer of up to `memory_size` = 5,000. Each training step samples a random mini-batch of 32. Consecutive days are highly correlated, and random sampling breaks that correlation. An early version of the RL notebook trained on the last 31 transitions *in order*, which defeated the purpose.
2. **Batched targets.** `exp_replay` does two batched forward passes (Q for s and for s') and one `train_on_batch` , instead of three Keras calls per sample. Only the taken action's Q-value is moved; the other outputs are set to their current values, so their error is zero:

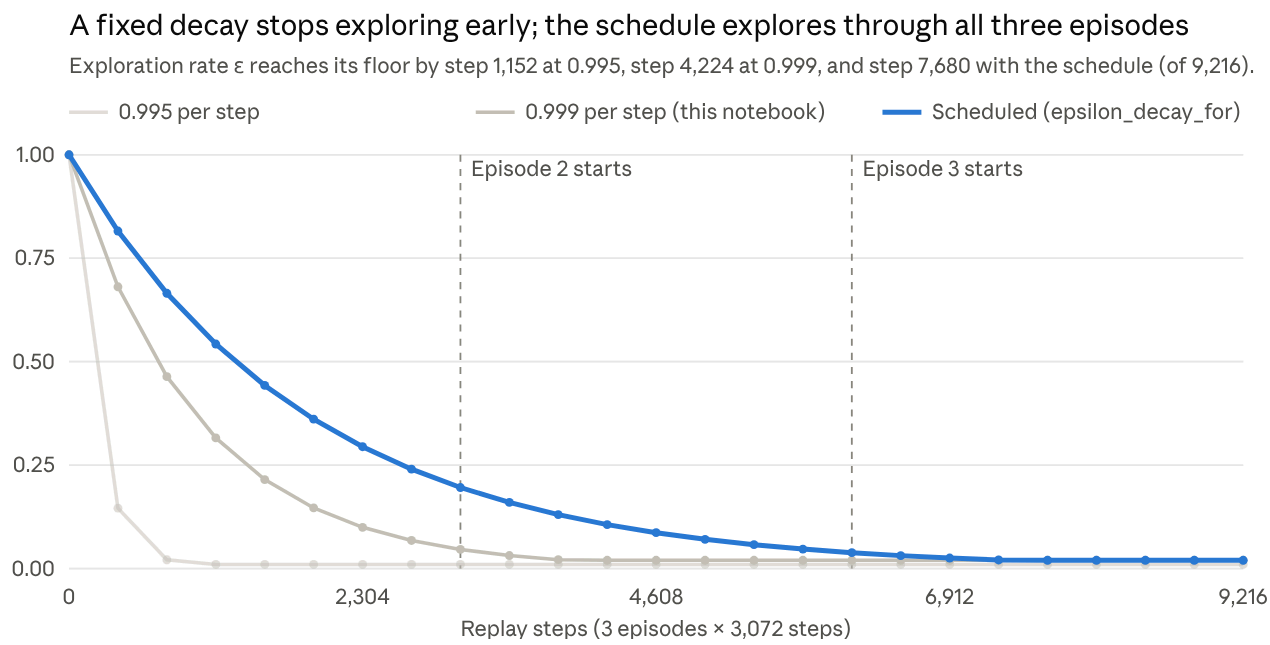
   ```
   targets = self.q_values(states)
   best_next = self._q(self.target_model or self.model,
   next_states).max(axis=1)
   targets[np.arange(batch_size), actions] = rewards + self.gamma *
   best_next * (1 - dones)
   loss = self.model.train_on_batch(states, targets)
   ```

3. **A target network** (Q with frozen weights $\bar{\theta}$). This is optional, and set with `target_update_every=N`. The target uses a lagged copy of the network, synced every N steps, so the network is not chasing its own moving prediction. The options notebook leaves it off ($\bar{\theta} = \theta$). Adding `target_update_every=200` to `AGENT_PARAMS` turns it on, as the AAPL RL notebook does.

Exploration: ε-greedy

With probability ε the agent takes a random action; otherwise the action with the highest Q. ε starts at 1 (pure exploration) and is multiplied by `epsilon_decay` after every replay step, down to `epsilon_min`. The decay rate decides how much of the data the agent ever explores. A fixed 0.995 hits the floor after about 900 steps, well inside the first episode. That was one cause of the AAPL agent's empty backtest. The options notebook uses 0.999. `func_lib.epsilon_decay_for(total_steps)` instead solves for the rate that reaches the floor after 80% of training:

```
\text{decay} = \left(\frac{\varepsilon_{\min}}
{\varepsilon_0}\right)^{1/(0.8\,N)}
```



Computed from the decay formulas · ε starts at 1.0; floors 0.01 (0.995 rule, original AAPL notebook) and 0.02 (this notebook); 3 episodes of 3,072 steps

With the 0.999 setting, the agent spends episodes 2 and 3 almost entirely greedy, repeating what it learned in episode 1. The schedule spreads exploration over all the training data.

In test mode (`test_mode=True`) ε is ignored: the agent is fully greedy and nothing is learned. That is what makes the out-of-sample test a fair evaluation of the learned policy.

Part 7 — The options book: actions, costs, margin, reward (cells 14–15)

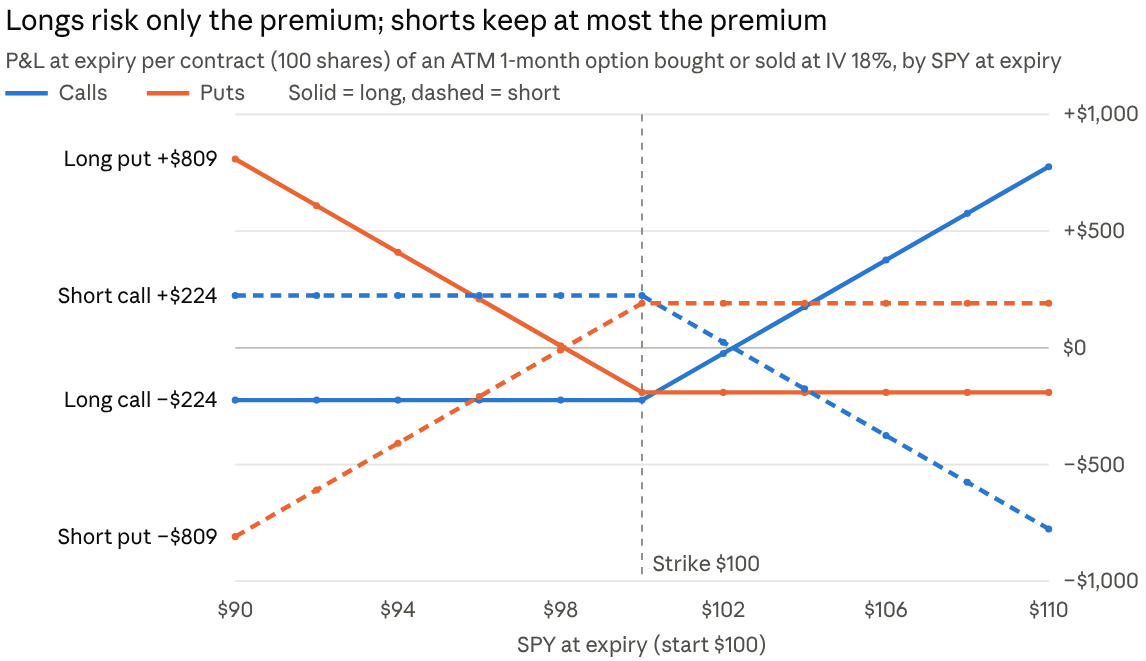
`run_options_episode` is the environment. It turns the agent's daily choice into trades, prices them with spreads, enforces margin and risk limits, marks the book to market, and returns the reward. Getting this realistic matters more than the network: an agent learns whatever the environment lets it get away with.

Actions and cash flows

Each trade is one contract (100 shares) of a European ATM option expiring in 21 trading days.

Action	Cash at entry	Exit (agent close / margin call)	Margin
0 hold	–	–	–
1 buy call / 2 buy put	pay ask = BS(IV + 1 pt) × 100	sell at bid = BS(IV − 1 pt)	none
3 sell call / 4 sell put	receive bid = BS(IV − 1 pt) × 100	buy back at ask = BS(IV + 1 pt)	Reg-T, see below
5 close oldest	–	FIFO: the oldest open position	released

Every day open positions are **marked at mid**, BS(IV), and at expiry they settle at intrinsic value. An action that cannot be done becomes a hold. That covers a full book (`max_positions` = 6), the short cap (`max_short_positions` = 3), too little free equity, a short blocked by the panic filter, or nothing to close.



Computed with func_lib.bs_price · premiums at IV 18%: call $2.24, put $1.91; S = K = 100, T = 21/252, r = 4%; spreads ignored

The chart shows why the sides behave so differently. A long option's loss is capped at its premium ($224 for the call at IV 18%). A short option's gain is capped at the premium, while its loss grows with the move: about $809 on a 10% drop for the short put.

Margin for naked shorts

```
\text{margin} = 100 \times \Big(V + \max\big(0.20\,S - \text{OTM},\
0.10\,S\big)\Big)
```

V is the option's current value. OTM is how far out of the money it is (K − S for a call, S − K for a put, floored at 0). **Example:** sell the ATM put above at the bid (IV 17%), $1.7925. Margin = 100 × (1.7925 + max(20 − 0, 10)) = **$2,179.25**, so about 12× the premium collected is tied up.

- **Free equity** = equity − margin in use. A buy needs free equity ≥ its premium; a sale needs free equity ≥ its new margin.
- **Margin call:** after each day's mark, while equity < margin in use, the oldest short is bought back at the ask, with exit reason `Margin call`.
- **Panic filter** (`PANIC_FILTER`): no *new* shorts when the trailing 21-day return is negative *and* IV is above its training-period 80th percentile. This follows the momentum-crash research (Daniel & Moskowitz): losses cluster after declines in high-vol states.

Reward and accounting

```
R_{t+1} = 100 \times \frac{E_{t+1} - E_t}{\text{capital}}, \qquad E_t =
\text{cash}_t + \sum_{\text{positions}} \text{sign}\times V_t \times 100
```

The reward is the daily change in marked equity, so the bid/ask spread is felt the moment a trade opens: buying at the ask and marking at mid books an immediate small loss. That discourages churning without any special penalty term. P&L is tracked so that, by construction, **final equity − starting capital = the sum of all trade P&L**. This held to the cent in every test run, and it caught a sign bug during development.

The state the agent sees

```
s_t = \Big[\ \underbrace{\mathrm{expit}(\Delta z_t)}_{\text{changes}},\
\underbrace{\mathrm{expit}(z_t)}_{\text{levels}},\
\underbrace{b_t}_{\text{book, 6 values}}\ \Big]
```

- **z** = the 12 features (Close, SMA_20, SMA_50, RSI, BB bands, sigma_f, iv, vrp, mu, call_edge, put_edge), standardized with a scaler **fitted on training data only**.
- **Changes**: the one-day differences through a sigmoid (`build_states`), as in the original RL notebook. 0.5 means no change.
- **Levels**: expit of the scaled values. Without them the agent could see that the edge *moved* but not whether it is high.
- **Book** `b` : the share of slots held as long call, long put, short call and short put; average time to expiry; margin utilization. Without these the policy cannot know what it holds. The AAPL agent lacked them, and its policy collapsed to buying every day.

The cells

Cell 14: states and agent.

1. Split `opt` by date: the first 70% for training.
2. Fit `StandardScaler` on training rows only.
3. Compute the panic flag and `short_allowed` per row.
4. Build states for the *whole* series in one pass, then slice, so the first test state continues from the last training days instead of being padded.
5. Bundle each period's arrays into the `md` dict the runner expects: dates, S, iv, r, sigma_f, mu, the edges, short_allowed.

6. Create `Agent(state_size = market features + 6, action_size = 6, ...)` and print the network summary.

Cell 15: training. For each of `EPISODES` = 3 passes over the training period, it calls `run_options_episode(..., train=True)`. Each step the agent acts, the transition is stored, and one replay step runs. After each pass the cell prints the per-position summary, plots the behaviour chart and loss curve, and saves `options_dqn_ep{e}.keras`. The behaviour chart has four rows: entries by position type, open positions (longs above zero, shorts below), margin used, and equity.

Part 8 — Code design: how the pieces fit and why

The notebook is thin on purpose: it sets parameters, calls `func_lib.py`, and shows results. The reusable maths, the environment, the agent and the charts live in the library, where every notebook in the project shares and tests them.

What lives where

func_lib section	Main functions	Used by this notebook for
1. Plotly theme	`set_plotly_theme`, `PALETTE`, `UP`, `DOWN`, `RANGE_SELECTOR`	the dark look of every chart (applied on import)
3. Time-series features	`download_ohlcv`, `build_ts_features`	SPY OHLCV; SMA, Bollinger, RSI features
5. RL core	`build_states`, `build_dqn`, `Agent`, `plot_losses`	state vectors, the network, the learner
6. Options & volatility	`bs_price`, `bs_greeks`, `gbm_paths`, `mc_option_price`, `gbm_expected_value`, `walk_forward_har/garch`, `forecast_metrics`, `performance_stats`, `stats_table_figure`	Parts 1–5 and the result tables
7. Options RL	`run_options_episode`, `short_margin_requirement`, `options_trade_statistics`, `plot_options_behavior`, `options_trade_analysis_figure`	the environment, tracking and dashboards

Design decisions, and why

Decision	Why
All settings in one config cell (cell 2)	An experiment is a one-line change, and the notebook's assumptions are visible in one place.
Vectorized NumPy everywhere (Black-Scholes over Series, `lfilter` GARCH, one-array GBM paths, `sliding_window_view` states)	Speed, and fewer index bugs: one formula, all days at once.
Strict no-look-ahead rules: walk-forward refits, train-only scaler, the HAR `start − horizon` cut, states built once then sliced	A backtest that peeks at the future is worthless. The literature review's "backtest hygiene" theme.

Decision	Why
Agent, environment and policy are separate (`run_options_episode(..., agent=None, policy=fn)`)	Benchmarks (GBM rule, random) run through the *same* environment, costs and limits as the DQN. The comparison is fair by construction.
Same `Agent` class as the AAPL RL notebook	One tested implementation; improvements such as the target network and ε scheduling reach both notebooks.
Episode results are plain dicts of arrays and DataFrames	Easy to inspect, plot, save and compare, with no hidden state.
Every trade logged with entry context (IV, forecast vol, edge, Greeks)	Results can be explained after the fact: why it won or lost, not just how much.
Lazy imports of keras, tqdm and yfinance inside functions	Notebooks that don't need them still import `func_lib`.
Verification built in (MC vs BS, Greeks vs finite differences, equity = Σ trade P&L)	Each piece is checked against an independent calculation. This caught a cash-sign bug in the short-selling code.
One shared dark Plotly theme	Every chart in the project reads the same way.

The core loop, in outline

```
for t in range(n - 1):
    action = policy(t, state) if policy else agent.act(state)      # epsilon-
greedy
    # open (if allowed: slots, short cap, free equity, panic filter) or close
the oldest
    # move to t+1: settle expiries, re-mark the book, force margin calls
    reward = 100 * (equity[t + 1] - equity[t]) / capital
    next_state = concat(market_states[t + 1], book_features(t + 1))
    if train:
        agent.remember(state, action, reward, next_state, done)
        agent.exp_replay(batch_size)                                  # one
gradient step
    state = next_state
```

The order inside a step matters. The trade happens at t's prices, *then* the market moves, *then* the reward is measured. So every action is judged only on what happened after it was taken.

Part 9 — Backtest, benchmarks and trade tracking (cells 16–21)

The trained agent is judged only on the 30% of dates it never trained on, with exploration off. It is compared against three benchmarks run through the same environment. Then every trade is broken down by position type.

The cells

Cell 16: out-of-sample test. It loads the last saved model into `Agent(..., test_mode=True)`, which is greedy and does no learning, and runs the test period. The same runner then plays two benchmark *policies* (functions of t and the state):

- **GBM-edge rule:** buy the option with the larger edge if it is above +5%; sell the one priced furthest above GBM value if that gap is over 15%; otherwise hold. Positions are held to expiry. It is a transparent, non-learning use of the same signal.
- **Random policy:** a uniformly random action each day. It shows what costs and limits alone do to an account. An agent that cannot beat it has learned nothing useful.

Cell 17: equity and statistics. It plots account equity and drawdown for the agent, the rule, the random policy and SPY buy-and-hold (scaled to the same capital). Then it tabulates `performance_stats` on daily equity returns: total return, CAGR, volatility, Sharpe, Sortino, max drawdown, Calmar, skewness and the share of positive days.

Cell 18: trade statistics. `options_trade_statistics` groups the trade log by **All**, then **Side** (Long, Short), **Type** (Call, Put) and **Position** (the four combinations). For each group it reports trade count, win rate, total and average P&L, average and median return, best and worst trade, profit factor, holding days, the share held to expiry, margin calls, average entry IV and average GBM edge. It runs for the agent and the rule.

Cell 19: the trade log. One row per closed trade: position, side, type, entry and exit dates, exit reason, strike, entry and exit spot, entry IV, forecast vol, GBM edge, entry premium, exit value, P&L ($), return % and holding days, plus the signed position Greeks at entry. It is saved to `options_agent_trade_log.csv` for your own analysis.

Cell 20: dashboards.

- a 2×2 trade analysis: cumulative realized P&L by position, the return distribution, return vs GBM edge at entry, and P&L by exit reason and position
- a monthly account-return heatmap (year × month)
- a bar chart of the actions chosen

Cell 21: today's signal. It builds today's state with an empty book and reads the six Q-values. It masks actions that are not available: closing with an empty book, and selling when the panic filter is on. Then it prints the best remaining action alongside spot, IV, HAR vol and both edges.

How to read the results

Question	Where to look	What a good answer looks like
Did the agent learn anything?	Cell 17: agent vs random	Clearly better Sharpe and drawdown than random, after costs
Is it better than the simple rule?	Cell 17: agent vs GBM-edge rule	If not, the network adds nothing over the signal itself
Where does the P&L come from?	Cell 18 by position; cell 20 cumulative P&L	Know which of the four positions earns it, e.g. short puts harvesting VRP
Is it tail-risk in disguise?	Skewness, worst trade, margin calls, max drawdown	Short-premium books show negative skew; check the 2020 and 2022 months on the heatmap
Is the edge real?	Cell 20: return vs GBM edge scatter	Higher edge at entry should line up with higher trade return
Is it trading at all?	Cell 20 action counts; cell 17	A policy stuck on one action points to a training problem (Part 11)

Part 10 — Next steps: optimizing the agent and upgrading the models

The notebook is a working baseline, not a finished strategy. This chapter lists the upgrades most likely to pay off. First the DQN agent: how it learns, what it is rewarded for, and what it can see. Then the pricing and forecasting models underneath it. Each item names the problem it fixes and where it plugs into `func_lib.py`.

A. Optimizing the DQN agent

Technique	Problem it fixes	Effort	Where it goes
Target network + Double DQN	Q-values overestimated by the max operator; unstable targets	Low	`Agent.exp_replay`
Action masking in targets	Bellman targets use actions that are impossible next step	Low	`run_options_episode`, `exp_replay`
Risk-aware reward (differential Sharpe, drawdown or CVaR penalty)	Reward ignores risk, so the agent can learn to sell tail risk	Low–medium	reward line in `run_options_episode`
n-step returns	One-day rewards say little about a 21-day trade	Medium	replay buffer
Prioritized experience replay	Rare, informative transitions (crashes) are sampled as often as dull ones	Medium	replay buffer
Dueling network	Hard to learn which states are good regardless of action	Medium	`build_dqn`
Richer state (book Greeks, regime features, recurrent memory)	Agent cannot see its net delta or vega, or anything beyond one day	Medium–high	state builder, `build_dqn`

Technique	Problem it fixes	Effort	Where it goes
Distributional RL (QR-DQN)	Expected value hides the fat left tail of short options	High	`build_dqn`, loss
Actor-critic (PPO, SAC) with continuous sizing	One-contract trades; no position-size decision	High	new agent class
Robust training protocol	One lucky seed, one regime, results tuned on the test set	Medium	the notebook

1. Target network and Double DQN

The max in the Bellman target picks whichever action's Q-value happens to be over-estimated, so errors compound upward. **Double DQN** (van Hasselt et al., 2016) lets the online network *choose* the next action and the target network *score* it:

```
y = r + \gamma\,(1-d)\;Q_{\bar\theta}\Big(s',\ \arg\max_{a'} Q_\theta(s', a')\Big)
```

In `exp_replay` this replaces one line. Turn on the target network first with `target_update_every=200` in `AGENT_PARAMS`:

```
q_next_online = self.q_values(next_states)                    # choose with the online net
q_next_target = self._q(self.target_model or self.model, next_states)
best = q_next_target[np.arange(batch_size), q_next_online.argmax(axis=1)]
targets[np.arange(batch_size), actions] = rewards + self.gamma * best * (1 - dones)
```

2. Action masking in the targets

The options book turns impossible actions into holds, but the target still takes the max over all six actions, including impossible ones. Store a valid-action mask with each transition and take the max over valid actions only: set masked Q-values to $-\infty$ before the max. Also pass the mask to `Agent.act(state, valid_actions)`, which already supports it. This removes a source of bias for free.

3. A reward that prices risk

The reward is today's P&L, so selling options looks great until the day it does not. Three standard fixes:

- **Differential Sharpe ratio** (Moody & Saffell, 2001): the per-step change in an exponentially weighted Sharpe ratio. The agent then maximizes risk-adjusted return directly, the lesson of the deep-momentum papers in the literature review.
- **Drawdown penalty:** $R' = R - \lambda \cdot \max(0, \text{drawdown increase})$.
- **CVaR or semivariance penalty:** subtract λ times the downside variance of recent rewards, which targets negative skew.

```
D_t = \frac{B_{t-1}\,\Delta A_t - \tfrac12 A_{t-1}\,\Delta B_t}{(B_{t-1} - A_{t-1}^2)^{3/2}}, \qquad A_t = A_{t-1} + \eta (R_t - A_{t-1}),\ B_t = B_{t-1} + \eta (R_t^2 - B_{t-1})
```

A and B are running estimates of the first and second moment of returns, $\eta \approx 0.01$. Feed D_t as the reward.

4. n-step returns and prioritized replay

- **n-step targets** add up n real rewards before bootstrapping: $y = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n \max Q(s_{t+n})$. With n ≈ 5–10 the agent links a trade to its outcome faster.
- **Prioritized replay** (Schaul et al., 2016) samples transition i with probability proportional to $|\delta_i|^\alpha$, where δ is the Bellman error. It corrects the bias with importance weights $w_i = (N p_i)^{-\beta}$. Crash days, the most informative and the rarest, then get replayed often enough to learn from.

Together with Double and Dueling DQN these are the core of **Rainbow** (Hessel et al., 2018), the standard strong DQN recipe.

5. Network and state upgrades

- **Dueling heads:** $Q(s,a) = V(s) + A(s,a) - \text{mean}_a A(s,a)$. The value of a state (high-VRP regime, say) is learned once and shared across actions.
- **Book Greeks in the state:** net delta, gamma, vega and theta of open positions, scaled by capital. The agent can then learn to hedge or to avoid stacking the same exposure.
- **Regime features:** VIX percentile, term-structure slope (VIX vs VIX3M), trailing drawdown, the panic flag itself.

- **Memory:** `WINDOW_SIZE > 1`, a recurrent DQN (LSTM), or attention as in the Momentum Transformer, so the agent sees trends and turning points rather than one day of changes.

6. Distributional RL and actor-critic

- **QR-DQN** learns quantiles of the return distribution instead of its mean. The policy can then act on the 5% quantile, a CVaR-style rule that suits a book that sells options.
- **PPO or SAC** with a continuous action, such as target vega exposure from −1 to +1 of a risk budget, replaces one-contract trades with real position sizing. Libraries such as Stable-Baselines3 can drive `run_options_episode` once it is wrapped as a Gymnasium environment.

7. A training protocol you can trust

1. **Three-way split:** train, *validation* (choose hyper-parameters and the best episode there), and test (look once).
2. **Walk-forward retraining:** retrain each year on an expanding window, as the volatility models already do.
3. **Seeds and ensembles:** train 5–10 seeds. Report the mean and spread, or vote across agents.
4. **Hyper-parameter search** (Optuna or a grid) on validation Sharpe: learning rate, γ, network width, decay schedule, cost level.
5. **Synthetic pre-training:** train on thousands of simulated GBM, Heston or bootstrapped paths, then fine-tune on history. This gives the agent far more crashes and regimes to learn from than 15 years of data.
6. **More realism before trusting results:** per-contract commissions, a strike and expiry grid (OTM puts, 1–3 months), and a delta-hedge action.

B. Intermediate to advanced pricing and forecasting models

GBM fails in three measurable ways: one volatility for all strikes (no skew), constant volatility over a trade, and no jumps. Each model below fixes one or more of these. Better volatility forecasts, in turn, feed directly into the GBM edge and the agent's state.

Pricing models, from intermediate to advanced

Model	Dynamics	Captures	Pricing method	Level
BS + fitted smile (SVI)	GBM per strike, IV read off a fitted surface	Skew and smile; arbitrage-free interpolation	Black-Scholes at IV(K, T)	Intermediate
Local volatility (Dupire)	$dS/S = \mu\,dt + \sigma(S, t)\,dW$	Fits today's whole surface exactly	PDE or Monte Carlo	Intermediate
Merton jump-diffusion	GBM + lognormal jumps at rate λ	Crash risk; short-dated skew	Poisson-weighted sum of BS prices	Intermediate
Heston	variance v follows a square-root process correlated with S	Stochastic vol, leverage effect, vol-of-vol	Characteristic function (COS or FFT); MC	Advanced
Bates	Heston + Merton jumps	Short- and long-dated skew together	Characteristic function	Advanced
SABR	stochastic vol on the forward (α, β, ρ, ν)	Smile per expiry; market standard in rates and FX	Hagan's closed-form IV approximation	Advanced
Rough Bergomi	log-variance driven by fractional noise, $H \approx 0.1$	ATM skew that steepens as expiry shortens; the "rough vol" facts	Monte Carlo (hybrid scheme)	Advanced
American exercise	any of the above	Early exercise of SPY options around dividends	Binomial tree; Longstaff-Schwartz regression MC	Intermediate
Deep calibration	a network learns model price = f(parameters)	Calibrates rough or Heston models in milliseconds	Trained NN surrogate	Advanced

SVI smile (Gatheral). Total implied variance w = σ²_IV T, as a function of log-moneyness k = ln(K/F), for each expiry:

```
w(k) = a + b\Big(\rho\,(k - m) + \sqrt{(k - m)^2 + \sigma^2}\Big)
```

Five parameters per expiry, fitted to market IVs, with no-butterfly and no-calendar-arbitrage checks. This is the most valuable first upgrade. It lets the agent trade OTM puts and calls at realistic prices instead of one ATM vol from VIX.

Dupire local volatility comes straight from the call-price surface C(K, T), with q = 0:

```
\sigma_{loc}^2(K, T) = \frac{\partial C/\partial T + rK\,\partial C/\partial
K}{\tfrac12 K^2\,\partial^2 C/\partial K^2}
```

Merton jump-diffusion. With jump intensity λ and log-jump size ~ N(μ_J, σ_J²), the price is a Poisson-weighted mix of Black-Scholes prices. It reuses `bs_price` directly:

```
def merton_price(S, K, T, r, sigma, lam, mu_j, sig_j, kind='call',
n_terms=40):
    k = np.exp(mu_j + 0.5 * sig_j ** 2) - 1          # mean relative jump
size
    lam_p, total = lam * (1 + k), 0.0
    for n in range(n_terms):
        sigma_n = np.sqrt(sigma ** 2 + n * sig_j ** 2 / T)
        r_n = r - lam * k + n * np.log(1 + k) / T
        weight = np.exp(-lam_p * T) * (lam_p * T) ** n / math.factorial(n)
        total += weight * bs_price(S, K, T, r_n, sigma_n, kind=kind)
    return total
```

Heston stochastic volatility:

```
dS_t = \mu S_t\,dt + \sqrt{v_t}\,S_t\,dW^S_t, \qquad dv_t = \kappa(\theta -
v_t)\,dt + \xi\sqrt{v_t}\,dW^v_t, \qquad d\langle W^S, W^v\rangle = \rho\,dt
```

κ is the speed of mean reversion, θ the long-run variance, ξ the vol of vol, and ρ < 0 the leverage effect that produces equity skew. Variance stays positive if 2κθ > ξ² (the Feller condition). Price with the characteristic function (the COS method of Fang & Oosterlee is fast and accurate), or simulate with full truncation:

```
for _ in range(n_steps):                                    # vectorized over
paths
    z1 = rng.standard_normal(n); z2 = rho * z1 + np.sqrt(1 - rho**2) *
rng.standard_normal(n)
    vp = np.maximum(v, 0)
    S *= np.exp((r - 0.5 * vp) * dt + np.sqrt(vp * dt) * z1)
    v += kappa * (theta - vp) * dt + xi * np.sqrt(vp * dt) * z2
```

Rough volatility (Gatheral, Jaisson & Rosenbaum; Bayer, Friz & Gatheral). Log-variance is driven by a Volterra process with Hurst H ≈ 0.1, far rougher than Brownian motion's 0.5. The model reproduces the observed ATM skew, which grows roughly like T^(H−1/2) as expiry shortens. It has no closed form, so it is priced by Monte Carlo (the hybrid scheme). It is usually calibrated with a neural-network surrogate (Horvath, Muguruza & Tomas, 2021).

Forecasting upgrades

Upgrade	Idea	Why it helps here
log-HAR, HAR-J, HARQ	Model log RV; add a jump component; let the daily coefficient shrink when RV is measured noisily (Bollerslev, Patton & Quaedvlieg, 2016)	Usually beats plain HAR on QLIKE
HAR-IV	Add VIX as a regressor	Combines market expectations with realized persistence
GJR or EGARCH, Realized GARCH	Asymmetric response to down moves; use RV inside GARCH	Captures the leverage effect GBM ignores
RFSV forecast	Predict log-vol with a fractional kernel, H ≈ 0.1	Matches or beats HAR in several studies (literature review)
ML on RV features	Gradient boosting or LSTM on HAR inputs, VIX, term structure, returns	Captures nonlinear regime effects; Bali et al. found nonlinear models help most
Implied-vol surface forecasting	Forecast SVI parameters or the whole surface, with no-arbitrage penalties (Chen & Zhang; Zhang et al.)	Forecasts the *prices* the agent trades, not just realized vol

Upgrade	Idea	Why it helps here
Density forecasts	GARCH-t, filtered historical simulation, quantile regression	Tail-aware ranges; fixes the calibration failures found in Part 4
Forecast combination	Weighted average of HAR, GARCH, RFSV and IV	Combinations are hard to beat out-of-sample

The RFSV log-vol forecast at horizon Δ weights past log-vol with a power-law kernel:

```
\mathbb{E}\big[\log\sigma_{t+\Delta}\,\big|\,\mathcal{F}_t\big] =
\frac{\cos(H\pi)}{\pi}\,\Delta^{H+1/2}\int_{-\infty}^{t} \frac{\log\sigma_s}
{(t - s + \Delta)(t - s)^{H+1/2}}\,ds
```

Judge every new forecaster with tests, not eyeballs. Use the Diebold-Mariano test for pairwise loss differences, Hansen's Model Confidence Set to find the group of statistically best models, and the Berkowitz test on PIT values for density forecasts. The notebook's `forecast_metrics` already computes the losses these tests need.

How they plug into the notebook

- **Market prices:** replace `bs_price(S, K, T, r, iv)` inside `run_options_episode` with Black-Scholes at an SVI-fitted IV(K, T) from real chain data. Strikes and expiries can then become actions.
- **The edge:** replace `gbm_expected_value` with a Heston or Bates *real-world* Monte Carlo value. That keeps the same edge = model value / market price − 1 idea, but with stochastic vol and jumps.
- **The volatility forecast:** swap `sigma_f` for the best model in an extended `forecast_metrics` table. Calibration in cell 12 shows whether the ranges improved.
- **The agent:** the new quantities (skew slope, Heston v_t, jump intensity, forecast quantiles) become extra state features.

Further reading

- van Hasselt, Guez & Silver (2016), Double DQN; Wang et al. (2016), Dueling DQN; Schaul et al. (2016), prioritized replay; Hessel et al. (2018), Rainbow; Dabney et al. (2018), QR-DQN; Schulman et al. (2017), PPO; Moody & Saffell (2001), differential Sharpe ratio

- Gatheral (2006), *The Volatility Surface*; Gatheral & Jacquier (2014), arbitrage-free SVI; Dupire (1994); Heston (1993); Merton (1976); Bates (1996); Hagan et al. (2002), SABR
- Bayer, Friz & Gatheral (2016), rough Bergomi; Horvath, Muguruza & Tomas (2021), deep learning volatility; Fang & Oosterlee (2008), COS method; Longstaff & Schwartz (2001)
- Bollerslev, Patton & Quaedvlieg (2016), HARQ; Diebold & Mariano (1995); Hansen, Lunde & Nason (2011), Model Confidence Set

Part 11 — Exercises, pitfalls and references

The fastest way to understand the notebook is to change one setting at a time and predict the result before you run it.

Exercises

- [ ] **Drift.** Set `MU_SHRINK = 0` and rerun cell 13. Edges now come only from implied vs forecast vol. Predict: call and put edges move together, both mostly negative.
- [ ] **Antithetic payoff.** In a scratch cell, compare `mc_option_price(..., antithetic=True)` for a call and for a straddle. Antithetic helps less for the straddle. Why? Hint: its payoff is symmetric in Z.
- [ ] **Calibration with a fat-tail fix.** Multiply the HAR vol by 1.2 in cell 12's z-score. Does 90% coverage reach 90%? What happens to the lower 5% tail?
- [ ] **Risk controls.** Turn `PANIC_FILTER` off and set `max_short_positions = 6`. Compare margin calls, worst trade, skewness and max drawdown in cells 17–18.
- [ ] **Costs.** Set `cost_vol = 0.02`. Which positions stop being profitable first?
- [ ] **Target network and ε.** Add `target_update_every=200` to `AGENT_PARAMS` and replace the fixed decay with `func_lib.epsilon_decay_for(EPISODES * len(states_train))`. Does test Sharpe become more stable across seeds?
- [ ] **Seeds.** Run with three values of `SEED`. If results swing widely, the learned policy is noise. Report the average, not the best.

Pitfalls

Symptom	Likely cause	Fix
Agent never trades, or only ever buys	Exploration ended too early; policy collapsed	Slower ε decay (`epsilon_decay_for`), more episodes, a different seed

Symptom	Likely cause	Fix
Great training P&L, poor test P&L	Overfitting one market regime	Fewer features, more episodes with replay, or train on several underlyings
Short-premium P&L looks too good	VIX overstates SPY ATM IV, so selling at VIX is too generous	Use real option-chain IVs, or subtract 1–2 vol points from `iv`
Huge losses in one month	Naked shorts through a crash	Panic filter on, lower `max_short_positions`, defined-risk spreads
Edges dominated by calls in bull runs	Momentum drift (`MU_SHRINK` high)	Lower the shrink; check edges with μ = r
Different results each run	Random replay and exploration	Fix `SEED`; compare over several seeds

What the model leaves out

- **Skew and real IVs:** one ATM implied vol from VIX for every option. Real chains have a volatility smile and term structure.
- **American exercise and dividends:** options are European on adjusted prices. Real SPY options are American and pay around dividends.
- **Constant-vol dynamics:** GBM holds σ fixed during a trade. Heston or rough-volatility models let it move.

Next steps

1. **Quick wins (a day):** target network + Double DQN, action-masked targets, a differential-Sharpe reward, several seeds with a validation split.
2. **Data realism (a week):** real option-chain data with skew (ORATS, OptionMetrics, CBOE DataShop), an SVI fit per expiry, OTM strikes and two expiries as actions, per-contract commissions.
3. **Risk and sizing (a week):** defined-risk trades (put spreads, iron condors) so a short position's loss is capped, and vega- or margin-based position sizing instead of one contract per trade.

4. **Better forecasts (a week):** log-HAR, HARQ and HAR-IV, RFSV as a volatility forecaster, and a forecast combination, compared with Diebold-Mariano and Model Confidence Set tests.
5. **Stochastic vol and jumps (2–3 weeks):** Heston and Bates pricing and calibration; a Bates-based edge.
6. **Advanced learning (ongoing):** prioritized n-step replay with dueling heads (Rainbow), then QR-DQN or PPO with vega-based sizing, pre-trained on simulated Heston paths and trained across several underlyings to reduce overfitting.
7. **Research frontier:** rough Bergomi dynamics with a neural calibration surrogate, and implied-vol surface forecasting as a state input.

References

- Black & Scholes (1973); Merton (1973): option pricing under GBM
- Corsi (2009), HAR-RV; Bollerslev (1986), GARCH; Patton (2011), QLIKE
- Gatheral, Jaisson & Rosenbaum (2018), *Volatility is Rough*
- Moskowitz, Ooi & Pedersen (2012), *Time Series Momentum*; Daniel & Moskowitz (2016), *Momentum Crashes*
- Bali et al. (2023), option return predictability; Mnih et al. (2015), DQN
- Project docs: *Literature Review* and *GBM Options RL Agent: Design Notes*